

Detection and segmentation

Dataset: COCO, 128 images

How to use these notes

The primary text for this lecture is Géron, Chapter 12. These notes are not a replacement for it and do not repeat it: they are the lecture’s own argument written out at length — the derivation done slowly, the numbers with the runs that produced them, and the reasoning between one slide and the next that a deck can only gesture at. Read the chapter for the method; read these for why the lecture makes the choices it does. Every figure quoted here is printed by this lecture’s notebook, and the course checks that mechanically rather than trusting it.

Contents

1	When a label is not an answer	2
1.1	Our corpus is 128 images	2
2	Derivation: scoring a box, scoring a detector	3
2.1	One quantity, four requirements	3
2.2	IoU as a loss, and why it fails	3
2.3	From one box to a whole detector	4
2.4	A mean of a mean of an area	5
3	Running a detector, and reading it	6
3.1	Check your own implementation against someone else’s	7
3.2	Suppression	7
4	Beyond boxes	8
5	The notebook	8
6	Exercises	8

1 When a label is not an answer

Three lectures of vision, all answering the same question: what is in this image? Neither a convolutional network nor a pretrained backbone can express *where* the object is, or *how many* there are. A photograph with three cyclists has one label and three answers.

A retail shelf audit puts it plainly: “*We photograph shelves. We need to know how many of each product are on each shelf, and where each one is, so that a picking route can be planned. A list of the categories present is no use to us.*”

	Classification	Detection
Question	what is in this picture	what is where, and how many
Output	one vector, length K	three arrays, length N
N is	fixed before you look	decided by the image and by two cut-offs
Wanted from pooling	invariance	equivariance

The last row is Lecture 12 with a consequence: *a network that has learned not to care where the object is cannot tell you where it is*. So the output stops being a label and becomes a set of boxes — which means the first thing we need is a new notion of what a correct answer is.

Failure condition — a box is four numbers, and which four is not agreed

$[x_1, y_1, x_2, y_2]$ is two opposite corners, used by torchvision. $[x, y, w, h]$ is a corner and a size, used by the COCO annotation file. $[cx, cy, w, h]$ is a centre and a size, used by the YOLO family.

You will hold the first two in the same notebook. Read $[100, 50, 20, 30]$ as corners and you get a box 20 wide when it should be 20 *starting at* 100 — a valid box, in the wrong place, with no error.

1.1 Our corpus is 128 images

Not COCO. 128 images out of the 5,000 in val2017, chosen as the numerically lowest image ids — a rule, not a selection. Full COCO with its training split is roughly 20 GB and is not downloaded anywhere in this course.

Quantity	Value
Images	128
Annotated objects, crowd regions excluded	898
Crowd regions, dropped	8
Distinct categories appearing	73 of 80
Objects per image — mean	7.02
Objects per image — median	5
Objects per image — range	1 to 29

Mean 7.02 against median 5: a long right tail, and one image alone carries 29 objects. And 39.0% of every annotated object in this corpus is a *person* — remember that when we start averaging over categories.

Every number in this lecture is measured on 128 images. When we quote a score, we quote the sample size beside it.

2 Derivation: scoring a box, scoring a detector

2.1 One quantity, four requirements

Before writing anything down, fix what it has to do: be 1 for coincident boxes and 0 for disjoint ones; punish a box for being too large; punish a box for being too small; and be dimensionless, so a cup and a sofa are on one scale.

Requirement 2 says the predicted area must be in the denominator, requirement 3 says the true area must be too, and requirement 4 says the numerator must be an area as well. There is essentially one candidate:

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}.$$

The second form is the one you compute — the union is never built — and subtracting the intersection is not an optimisation: add the two areas and you have counted the overlap twice.

A box covering the whole image scores 0.086 on our corpus: always right about something and never about anything in particular.

IoU does three different jobs

Matching: does a detection count as the same object as an annotation. *Suppression*: are two detections duplicates of each other. *Training*: as the loss a regression head descends. The first two use IoU as a comparison. Only the third needs it to have a derivative, and that is where the derivation has its teeth.

2.2 IoU as a loss, and why it fails

Two boxes, each 100 by 100. Fix the first at the origin, slide the second right by d pixels, and record the value and the gradient. Synthetic on purpose — nothing here depends on a dataset, so the conclusion is a property of the formula rather than of COCO.

d	IoU	$\partial \text{IoU} / \partial d$	GIoU	$\partial \text{GIoU} / \partial d$
0	1.000	0.000	1.000	0.000
50	0.333	-0.0089	0.333	-0.0089
90	0.053	-0.0055	0.053	-0.0055
150	0.000	0.000	-0.200	-0.0032
300	0.000	0.000	-0.500	-0.0013

Rows four and five are the point: two boxes 150 pixels apart and two boxes 300 pixels apart are, to IoU, *exactly equally wrong*.

Why the gradient is exactly zero, not merely small

For $d \geq 100$ the overlap width is $\max(0, 100 - d) = 0$, so

$$|A \cap B| \equiv 0 \implies \text{IoU} \equiv 0 \implies \frac{\partial \text{IoU}}{\partial d} \equiv 0.$$

The clamp makes the function *constant* on the whole disjoint region, not merely small there — and a constant has no descent direction. Not a weak one: none.

This is a property of the formula, and no optimiser, learning rate or initialisation repairs it. A proposal that starts anywhere outside the object it should find receives the same loss and the same zero gradient wherever it is, because nothing in the loss knows which way the object lies. Which is why detectors are not trained on IoU alone, and why the coordinate regression they *are* trained on needs anchors close enough to overlap in the first place.

The repair is to put the empty space back in. With C the smallest box containing both,

$$\text{GIoU}(A, B) = \text{IoU}(A, B) - \frac{|C| - |A \cup B|}{|C|}.$$

When the boxes are disjoint and move apart, $|C|$ grows and $|A \cup B|$ does not, so the penalty grows. CIoU adds two more terms — a centre distance normalised by the enclosing diagonal, and an aspect-ratio disagreement — and converges faster because it moves the centre directly rather than shrinking a gap.

2.3 From one box to a whole detector

IoU scores one pair. A detector emits 4,419 boxes over 128 images, most of them wrong, all carrying a score. Fixing a score threshold gives one precision and one recall — and throws away everything the ranking knew.

A detector is a ranking. Score it as one. Fix a class and an IoU threshold, sort every detection of that class over the whole corpus by score, and walk down the list: for each detection find the best *unmatched* annotation in the same image, and if that IoU clears the threshold it is a true positive and the annotation is now matched. That word *unmatched* is what makes a second box on the same bottle a false positive rather than a second success.

Precision and recall are unchanged from Lecture 3, computed cumulatively after each detection — so a corpus of 1,209 person detections gives 1,209 points. The recall denominator is fixed, so recall is non-decreasing; precision has both a numerator and a denominator moving.

Every step accounted for, exactly

Walking the person ranking one detection at a time:

	Steps
the detection is a false positive — precision falls	899
a true positive and precision was below 1 — precision rises	254
a true positive and precision was already 1 — no change	55

899 is exactly the number of false positives, and $254 + 55 = 309$ is the 310 true positives less the one at the top of the ranking, which has no step above it. The 55 flat steps are one contiguous run: the top 56 person detections in the whole corpus are all correct.

An exact classification of all 1,208 steps, in the same form as Lecture 3's.

Lecture 3 promised the repair: average precision is defined using the maximum precision at or above each recall level,

$$p_{\text{env}}(r) = \max_{\tilde{r} \geq r} p(\tilde{r}),$$

non-increasing by construction. Why a maximum rather than a smoothing? It has an operational reading — if you will accept recall r , here is the best precision available at that recall or better; it is monotone by construction, so the area under it does not depend on where you sampled; and it is one line of code with no parameters.

$$\text{AP} = \int_0^1 p_{\text{env}}(r) \, dr = \sum_k (r_k - r_{k-1}) p_{\text{env}}(r_k).$$

The sum only has terms where recall actually changed, which is exactly at the true positives — and *no threshold appears anywhere*. That is the whole point: the number does not depend on a cut-off nobody chose.

person, IoU at least 0.5	Value
Annotated instances	350
Detections in the ranking	1,209
True positives	310
Highest recall reached	0.886
AP	0.751

Recall stops at 0.886 because 40 annotated people are never detected at all, at any score. No threshold recovers them, and AP is honest about that.

2.4 A mean of a mean of an area

The first mean is over classes: $\text{mAP}(t)$ averages $\text{AP}_c(t)$ over the classes present. Unweighted — a class with one annotation counts as much as person with 350. We average over the 73 classes appearing in our 128 images; the other 7 have no annotations here. *Which classes are in the mean is a decision, it changes the number, and papers do not always say which one they made.* Thirteen classes score exactly 1.000, and they are the ones with one or two instances in the whole corpus.

The second mean is over IoU thresholds, averaging the ten values from 0.50 to 0.95. At 0.50 a box that half-covers the object counts as correct; at 0.95 it must be very nearly exact. That average is COCO’s headline number and what “mAP” means unqualified in a modern paper — and on our corpus AP runs from 0.659 at the loosest threshold to 0.040 at the tightest, with one number in the middle standing for both.

Our detector, on 128 images	Value
mAP at IoU 0.50	0.659
mAP at IoU 0.75	0.475
mAP averaged over 0.50 to 0.95	0.439

Three collapses, one number. Each throws away a different thing, and none is recoverable from the result.

3 Running a detector, and reading it

The specification first: 128 JPEG files and one annotation file in; per image, boxes in corner form, category ids, scores, and one integer count out; nothing trained; and the *check* — every predicted box lies inside the image, scores are non-increasing, and the count equals the number of boxes kept. That last row is the one people skip, and it is the one that catches a corner-versus-size mix-up in a line.

Convert once, at the edge, and assert it

COCO gives x, y, w, h ; everything else here is corners. The conversion happens once, on load, and then:

```
assert (b[:, 2] >= b[:, 0]).all(), "x2 < x1 - you read w as x2"
assert (b[:, 3] >= b[:, 1]).all(), "y2 < y1 - same bug"
```

Reading w as x_2 gives $x_2 < x_1$ for every box wider than its own left edge, which is almost all of them. Two lines catch it; nothing else will.

One photograph returns 88 boxes — not 88 objects. The three arrays share an index, and they arrive sorted by score descending, which is a thing to assert rather than assume.

Failure condition — there is already a cut-off, and it is not yours

`box_score_thresh` defaults to 0.05, so anything below is never returned at all. `box_nms_thresh` defaults to 0.5, so near-duplicates are removed before you see them. `box_detections_per_img` defaults to 100 — and our busiest image returns exactly 100 boxes, so that cap is not hypothetical.

Reviewer question 5, three times over.

The count is a function of the threshold: move the cut-off and the answer to the stakeholder’s question changes by a factor of ten. We adopt 0.5, because it is the conventional reporting threshold *and because it was chosen before looking at the error curve*. The minimum of that curve sits at 0.75 with an MAE of 1.92, and adopting it would be choosing a hyperparameter on

the data we then report — Lecture 2's error in a new costume. So it is quoted as a diagnostic and not as a result.

At score at least 0.5	Value
Count MAE	3.00
Signed count error	+2.41
Images counted exactly right	27 of 128
Images with too many boxes	83
Images with too few	18

Two numbers, not one: the absolute error says how far off, the signed error says which way — and the cloud sits above the diagonal, so the failure mode is too many boxes. 3.00 against a baseline of 6.02, so the model halves the error of a system that never looks at the image.

3.1 Check your own implementation against someone else's

We derived AP here and implemented it ourselves, which is exactly the situation in which a result should not be believed.

mAP on our 128 images	Ours	Reference evaluator
at IoU 0.50	0.659	0.660
at IoU 0.75	0.475	0.476
averaged 0.50 to 0.95	0.439	0.440

Largest disagreement 0.0014, against the official COCO evaluator run on the same predictions by a library we did not write.

And now compare 128 images against 5,000

Our averaged mAP is 43.9 on 128 images. torchvision reports 37.0 for *these same weights* on all 5,000.

6.9 points higher, and nothing about the detector changed. The difference is the sample: 128 images, many of them easy, and 73 classes several of which have a single instance that happened to be found. This is what a small evaluation set does — and it does it in the flattering direction more often than not.

3.2 Suppression

Without suppression the detector proposes 231.7 candidate boxes per image, against 23.6 after suppression at IoU 0.5, and 7.02 actual objects. *The detector you ran had already thrown away nine tenths of its own output before you saw it, using IoU, at a threshold you did not set.*

Use `batched_nms` rather than `nms`: a bottle overlapping a glass must not suppress it, because they are different classes and both are true. And the default of 0.5 happens to be near the best value on our corpus — a fact about this corpus, not a law.

4 Beyond boxes

A box was always an approximation. A bicycle's box is mostly not bicycle; a person sitting cross-legged fills perhaps half of theirs; and for a picking arm "somewhere in this rectangle" may not be good enough.

Semantic segmentation gives one class label per pixel, so two people standing together are one person region and you cannot count them. The architectural problem is that pooling destroyed the resolution the answer needs — Lecture 12's invariance arriving as a cost — and the repair is upsampling with skip connections that recover detail from the earlier, higher-resolution layers.

Instance segmentation gives one binary mask per detected object. Mask R-CNN is Faster R-CNN with one extra head predicting a small mask inside each proposed box, and it is two lines away from what you already ran. Only that output can answer "how many".

Scoring a mask is the same argument again: mask AP replaces box IoU with mask IoU, intersecting pixel sets instead of rectangles, and everything above — the ranking, the envelope, the area, the two means — is unchanged. torchvision reports 34.6 mask mAP for these weights against 37.9 box mAP on the full 5,000, which is what "a mask is harder than a box" means numerically.

And detection on video is not tracking. Run the detector on every frame and you have boxes, not the same bottle in frame 2 as in frame 1. Tracking asks for a persistent identity, which turns each frame into an assignment problem — and the cheapest similarity between a track's predicted box and a new detection is IoU, so the derivation pays for itself a third time. It also fails the same way: an object occluded for ten frames reappears where its predicted box does not overlap, IoU is zero, the assignment finds nothing, and the object is issued a new identity.

5 The notebook

What to change

Move one predicted box further and further from its match, and watch IoU sit at exactly zero the whole way. That flat zero is the vanishing gradient the derivation predicts — and seeing it as a constant rather than as a small number is the difference between the two answers to the exam question.

6 Exercises

1. Two boxes are disjoint and moved further apart. State what IoU and its derivative do, and why no learning rate repairs it. [5]
2. An IoU implementation without a clamp is tested on overlapping boxes only and passes. Give the input that breaks it, and the property test that would have caught it. [5]
3. Average precision is defined with a maximum inside it. State what that maximum repairs, and which earlier lecture proved the problem. [4]

4. A detector is evaluated on a corpus in which 30 of the classes present have exactly one annotated instance each, and reports mAP 0.72 at IoU 0.5. Give two distinct reasons the number may be optimistic, and one thing you would report alongside it. [3]
5. Explain why semantic segmentation cannot answer “how many”, and name the architectural property that causes it. [4]

Summary

A label is not an answer when the question is *where* and *how many*, and a network trained to be invariant to position cannot supply either. IoU is the essentially unique quantity meeting the four requirements — and it is exactly zero for every disjoint pair, so as a loss it offers no gradient however far apart the boxes are. Not a small one: none, because the clamp makes it constant. GIoU and CIoU repair that by adding a term that keeps moving.

Average precision is built by matching a ranking against annotations, and its definition contains a maximum for one reason: precision is not monotone, which Lecture 3 proved, and the envelope is the repair. Counted on the person ranking, 899 steps down and 254 up and 55 flat account for all 1,208 exactly. mAP then averages over classes and over ten IoU thresholds — a mean of a mean of an area, where our own AP runs from 0.659 to 0.040 across the thresholds it averages.

Our implementation agrees with the official evaluator to 0.0014, which is why it can be quoted. And it reports 43.9 where torchvision reports 37.0 for the same weights: 6.9 points, entirely from evaluating on 128 images instead of 5,000, in the flattering direction.